

Sample-Based Krylov Quantum Diagonalization (SKQD)

Sample-Based Krylov Quantum Diagonalization (SKQD) is a hybrid quantum-classical algorithm that combines the theoretical guarantees of Krylov Quantum Diagonalization (KQD) with the practical efficiency of sample-based methods. Instead of expensive quantum measurements to compute Hamiltonian matrix elements, SKQD samples from quantum states to construct a computational subspace, then diagonalizes the Hamiltonian within that subspace classically.

Why SKQD?

Traditional quantum algorithms like VQE face several fundamental challenges:

- **Optimization complexity:** VQE requires optimizing many variational parameters in a high-dimensional, non-convex landscape
- **Measurement overhead:** Computing expectation values $\langle \psi(\theta) | H | \psi(\theta) \rangle$ requires many measurements for each Pauli term
- **Barren plateaus:** Optimization landscapes can become exponentially flat, making gradient-based optimization ineffective
- **Parameter initialization:** Poor initial parameters can lead to local minima far from the global optimum

SKQD addresses these fundamental limitations:

- **No optimization required:** Uses deterministic quantum time evolution instead of variational circuits
- **Provable convergence:** Theoretical guarantees based on the Rayleigh-Ritz variational principle
- **Measurement efficient:** Only requires computational basis measurements (Z-basis), the most natural measurement on quantum hardware
- **Noise resilient:** Can filter out problematic measurement outcomes and handle finite sampling
- **Systematic improvement:** Increasing Krylov dimension monotonically improves ground state estimates

- **Hardware friendly:** Time evolution circuits are more amenable to near-term quantum devices than deep variational ansätze

Understanding Krylov Subspaces

What is a Krylov Subspace?

A **Krylov subspace** $\mathcal{K}^r$ of dimension r is the space spanned by vectors obtained by repeatedly applying an operator A to a reference vector $|\psi\rangle$:

$$\mathcal{K}^r = \text{span}\{|\psi\rangle, A|\psi\rangle, A^2|\psi\rangle, \dots, A^{r-1}|\psi\rangle\}$$

The SKQD Algorithm

The key insight of SKQD is that we can: 1. **Generate** Krylov states $U^k|\psi\rangle$ using quantum time evolution 2. **Sample** from these states to get computational basis measurements 3. **Combine** all samples to form a computational subspace 4. **Diagonalize** the Hamiltonian within this subspace classically

This approach is much more efficient than computing matrix elements via quantum measurements!

```
[1]: import cudaq
import numpy as np
import cupy as cp
import pickle

from skqd_src.pre_and_postprocessing import *

from scipy.sparse import csr_matrix as sp_csr_matrix
from scipy.sparse.linalg import eigsh as sp_eigsh

from cupyx.scipy.sparse import csr_matrix as cp_csr_matrix
from cupyx.scipy.sparse.linalg import eigsh as cp_eigsh

if cudaq.num_available_gpus() > 0 and cudaq.has_target("nvidia"):
    cudaq.set_target("nvidia")
else:
    cudaq.set_target("qpp-cpu")

cudaq.set_random_seed(42)
np.random.seed(43)
cp.random.seed(44)

print("Using:", cudaq.__version__)
```

```
/usr/local/lib/python3.12/dist-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found.
Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

Problem Setup: 22-Qubit Heisenberg Model

We demonstrate SKQD on a 1D Heisenberg spin chain with 22 qubits:

$$H = \sum_i (J_x \sigma_i^x \sigma_{i+1}^x + J_y \sigma_i^y \sigma_{i+1}^y + J_z \sigma_i^z \sigma_{i+1}^z)$$

```
[2]: num_spins = 22

if num_spins >= 63:
    raise ValueError(f"Vectorized implementation of postprocessing supports max 62 qubits due to int64 packing. Requested: {num_spins}")

shots = 100_000

total_time_evolution = np.pi
num_trotter_steps = 8
dt = total_time_evolution / num_trotter_steps

max_k = 12 # largest k for U^k

Jx, Jy, Jz = 1.0, 1.0, 1.0
H = create_heisenberg_hamiltonian(num_spins, Jx, Jy, Jz)
exact_ground_state_energy = -38.272304 # Computed via exact diagonalization
hamiltonian_coefficients, pauli_words = extract_coeffs_and_paulis(H)
hamiltonian_coefficients_numpy = np.array(hamiltonian_coefficients)
```

Krylov State Generation via Repeated Evolution

For SKQD, we generate the Krylov sequence:

$$|\psi_0\rangle, U|\psi_0\rangle, U^2|\psi_0\rangle, \dots, U^{k-1}|\psi_0\rangle$$

where $U = e^{-iHT}$ is approximated via Trotter decomposition.

Implementation Strategy: 1. Start with reference state $|\psi_0\rangle$ 2. Apply Trotter-decomposed time evolution k times for $U^k|\psi_0\rangle$ 3. Measure each Krylov state in computational basis 4. Accumulate measurement statistics across all Krylov powers

```
[3]: @cudaq.kernel
def quantum_krylov_evolution_circuit(
    num_qubits: int,
    krylov_power: int,
    trotter_steps: int,
    dt: float,
    H_pauli_words: list[cudaq.pauli_word],
    H_coefs: list[float]):

    """
    Generate Krylov states via repeated time evolution.

    Args:
        num_qubits: Number of qubits in the system
        krylov_power: Power k for computing  $U^k|\psi\rangle$ 
        trotter_steps: Number of Trotter steps for each U application
        dt: Time step for Trotter decomposition
        H_pauli_words: Pauli decomposition of Hamiltonian
        H_coefs: Coefficients for each Pauli term

    Returns:
        Measurement results in computational basis
    """

    qubits = cudaq.qvector(num_qubits)

    #Prepare Néel state as reference |010101...>
    for qubit_index in range(num_qubits):
        if qubit_index % 2 == 0:
            x(qubits[qubit_index])

    for _ in range(krylov_power):                                #applies  $U^k$ 
        where U = exp(-iHT)

            for _ in range(trotter_steps):                        #applies  $\exp(-iHT)$ 

                for i in range(len(H_coefs)):                    #applies  $\exp(-iHdt)$ 

                    exp_pauli( -1 * H_coefs[i] * dt, qubits, H_pauli_words[i]) #applies  $\exp(-ihdt)$ 

    mz(qubits)
```

Quantum Measurements and Sampling

The Sampling Process

For each Krylov power $k = 0, 1, 2, \dots, K - 1$: 1. **Prepare** the state $U^k|\psi_0\rangle$ using our quantum circuit 2. **Measure** in the computational basis many times 3. **Collect** the resulting bitstring counts

The key insight: these measurement outcomes give us a statistical representation of each Krylov state, which we can then use to construct our computational subspace classically.

```
[4]: all_measurement_results = []

for krylov_power in range(max_k):
    print(f"Generating Krylov state U^{krylov_power}...")

    sampling_result = cudaq.sample(
        quantum_krylov_evolution_circuit,
        num_spins,
        krylov_power,
        num_trotter_steps,
        dt,
        pauli_words,
        hamiltonian_coefficients,
        shots_count=shots)

    all_measurement_results.append(dict(sampling_result.items()))

cumulative_results = calculate_cumulative_results(all_measurement_results)
```

```
Generating Krylov state U^0...
Generating Krylov state U^1...
Generating Krylov state U^2...
Generating Krylov state U^3...
Generating Krylov state U^4...
Generating Krylov state U^5...
Generating Krylov state U^6...
Generating Krylov state U^7...
Generating Krylov state U^8...
Generating Krylov state U^9...
Generating Krylov state U^10...
Generating Krylov state U^11...
```

Classical Post-Processing and Diagonalization

Now comes the classical part of SKQD: we use our quantum measurement data to construct and diagonalize the Hamiltonian within each Krylov subspace.

1. **Extract basis states** from measurement counts
2. **Project Hamiltonian** onto the computational subspace spanned by these states
3. **Diagonalize** the projected Hamiltonian classically
4. **Extract ground state energy estimate**

Matrix Construction Details

The core of SKQD is constructing the effective Hamiltonian matrix within the computational subspace:

1. **Computational Subspace Formation:** From quantum measurements, we obtain a set of computational basis states $\{|s_1\rangle, |s_2\rangle, \dots, |s_d\rangle\}$ that spans our approximation to the Krylov subspace.
2. **Matrix Element Computation:** For each Pauli term P_k in the Hamiltonian with coefficient h_k :

$$H = \sum_k h_k P_k$$

We compute matrix elements $\langle s_i | P_k | s_j \rangle$ by applying the Pauli string P_k to each basis state $|s_j\rangle$.

3. **Effective Hamiltonian:** The projected Hamiltonian becomes:

$$H_{\text{eff}}[i, j] = \sum_k h_k \langle s_i | P_k | s_j \rangle$$

We demonstrate two eigenvalue solver approaches below: an explicit CSR sparse matrix and a matrix-free Lanczos iteration.

Approach 1: GPU-Vectorized CSR Sparse Matrix

We build the projected Hamiltonian as a CuPy CSR sparse matrix on the GPU and solve with

```
cupyx.scipy.sparse.linalg.eigsh
```

```
[5]: csr_energies = []

for k in range(1, max_k):

    cumulative_subspace_results = cumulative_results[k]
    basis_states = get_basis_states_as_array(cumulative_subspace_results, num_spins)
    subspace_dimension = len(cumulative_subspace_results)
    assert len(cumulative_subspace_results) == basis_states.shape[0]

    # #slower non-vectorized CPU implementation with CSR matrix input
    # matrix_rows, matrix_cols, matrix_elements = projected_hamiltonian_cpu(basis_states,
    pauli_words, hamiltonian_coefficients_numpy)
    # projected_hamiltonian = sp_csr_matrix((matrix_elements, (matrix_rows, matrix_cols)),
    shape=(subspace_dimension, subspace_dimension))
    # eigenvalue = sp_eigsh(projected_hamiltonian, return_eigenvectors=False,
    **eigenvalue_solver_options)

    v0_np = np.random.random((subspace_dimension,)).astype(np.float64)
    v0_np = v0_np / np.linalg.norm(v0_np)
    v0 = cp.asarray(v0_np, dtype=cp.complex128)

    vectorized_pj_on_gpu = True
    matrix_rows, matrix_cols, matrix_elements = vectorized_projected_hamiltonian(
        basis_states, pauli_words, hamiltonian_coefficients_numpy, vectorized_pj_on_gpu)

    projected_hamiltonian = cp_csr_matrix(
        (matrix_elements, (matrix_rows, matrix_cols)),
        shape=(subspace_dimension, subspace_dimension))

    eigenvalue = cp_eigsh(projected_hamiltonian, return_eigenvectors=False,
        k=1, which='SA', v0=v0)

    csr_energies.append(np.min(eigenvalue).item())
```

Approach 2: Matrix-Free Lanczos via `distributed_eigsh`

Instead of explicitly constructing the projected Hamiltonian as a sparse matrix, we can apply the Hamiltonian as a linear operator during Lanczos iteration. This matrix-free approach avoids storing the full sparse matrix and naturally extends to multiple GPUs (demonstrated in the multi-GPU section below).

```
[6]: lo_energies = []

for k in range(1, max_k):

    cumulative_subspace_results = cumulative_results[k]
    basis_states = get_basis_states_as_array(cumulative_subspace_results, num_spins)
    subspace_dimension = len(cumulative_subspace_results)

    v0_np = np.random.random((subspace_dimension,)).astype(np.float64)
    v0_np = v0_np / np.linalg.norm(v0_np)
    v0 = cp.asarray(v0_np, dtype=cp.complex128)

    ham_data = prepare_hamiltonian_data(basis_states, pauli_words,
    hamiltonian_coefficients_numpy)
    cp.get_default_memory_pool().free_all_blocks()
    ncv = min(20, subspace_dimension - 1)

    eigenvalues, = distributed_eigsh(
        ham_data, rank=0, size=1, nccl_comm_obj=None,
        k=1, which='SA', ncv=ncv, v0_full=v0,
        tile_size=32, return_eigenvectors=False)

    eigenvalue = eigenvalues[0]
    eigenvalue_real = float(cp.asnumpy(eigenvalue).real)
    lo_energies.append(eigenvalue_real)
```

Results Analysis and Convergence

Let's visualize our results and analyze how SKQD converges to the true ground state energy.

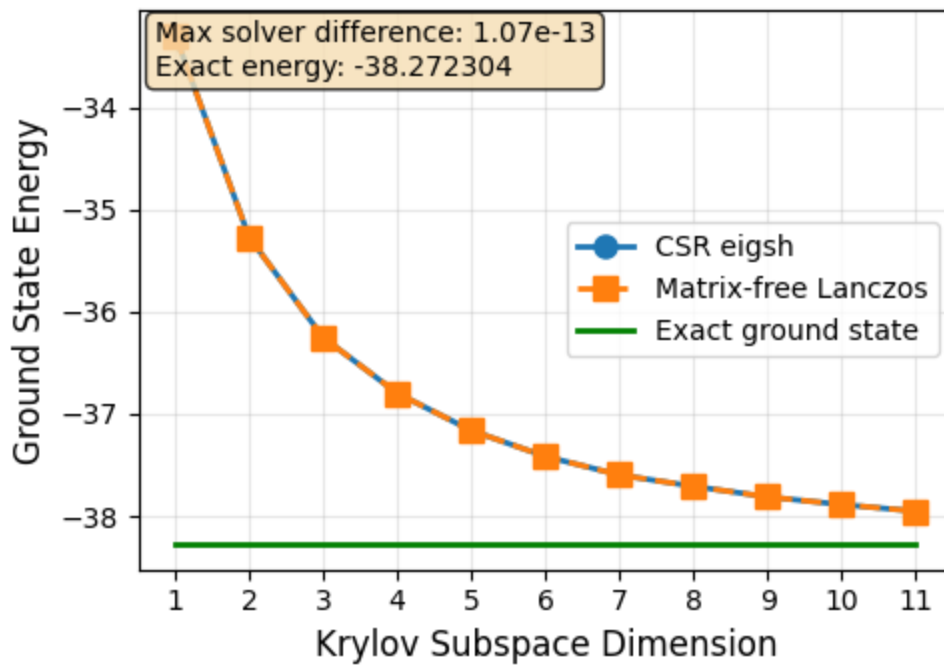
What to Expect:

- **Monotonic improvement:** Each additional Krylov dimension should give a better (lower) energy estimate
- **Convergence:** The estimates should approach the exact ground state energy
- **Diminishing returns:** Later Krylov dimensions provide smaller improvements

The exact ground state energy for our selected Hamiltonian was computed earlier via classical exact diagonalization and will be used as the reference for comparison with SKQD results.

```
[7]: plot_skqd_convergence(csr_energies, lo_energies, range(1, max_k), exact_ground_state_energy)
```

SKQD Ground State Energy Convergence

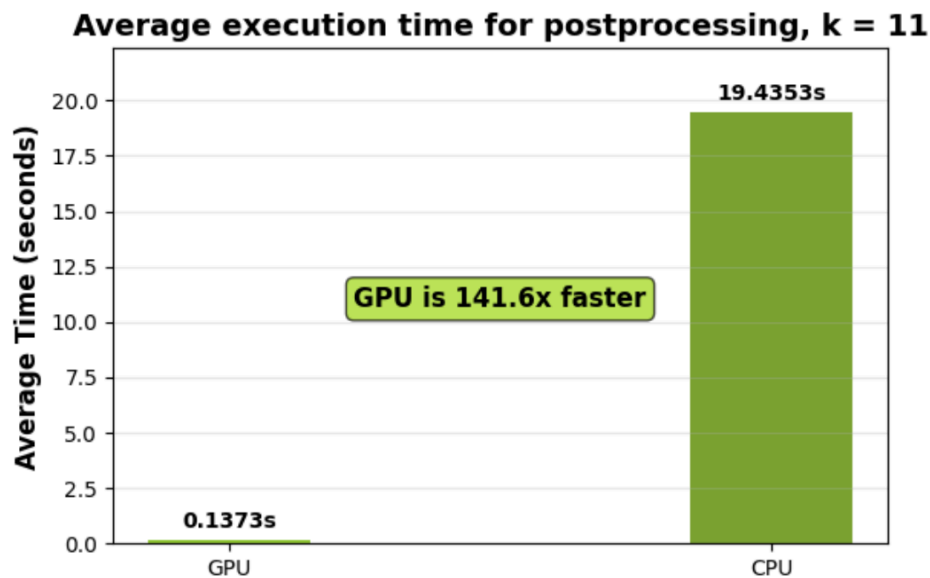


Postprocessing Acceleration: CSR matrix approach, single GPU vs CPU

The data in the plot below was gathered by averaging over 5 runs on a NVIDIA H100 GPU and an Intel Xeon Platinum 8480CL CPU with 224 threads. The only thing that changed between the 2 data points was `vectorized_pj_on_gpu = True` and `vectorized_pj_on_gpu = False`

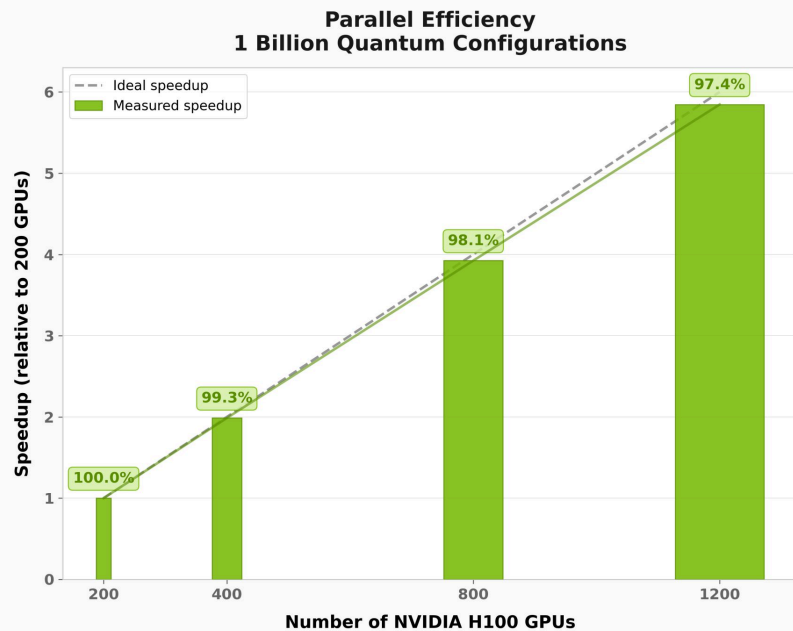
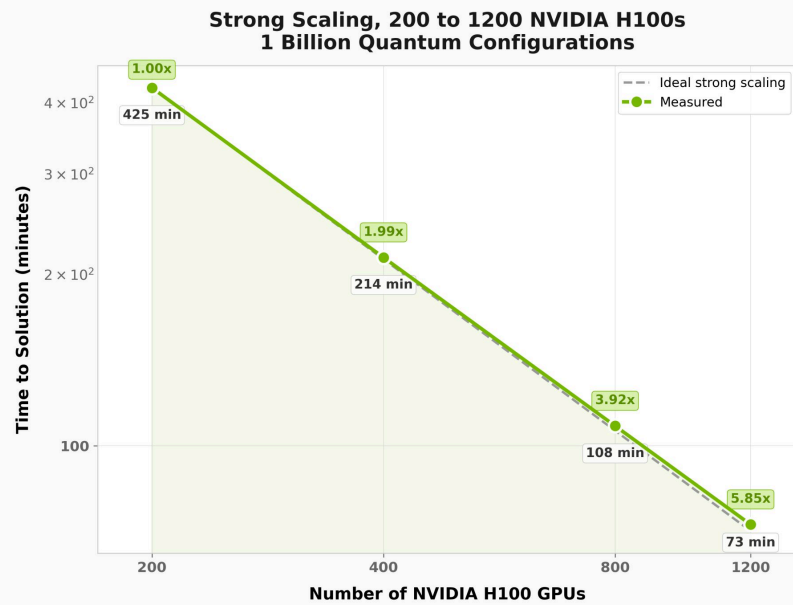
This substantial acceleration comes from: - **Parallel Pauli string evaluation** across thousands of basis states simultaneously - **Vectorized matrix element computation** leveraging CUDA cores - **GPU-optimized sparse linear algebra** via cuSPARSE library for eigenvalue solving

The speedup becomes increasingly critical as the Krylov dimension grows, since the computational subspace dimension (and thus the matrix size) scales exponentially with k . For higher k values, GPU acceleration transforms previously intractable postprocessing into feasible computation times.



Postprocessing Scale-Up and Scale-Out: Linear Operator Approach, Multi-GPU Multi-Node

The plots below were generated using `distributed_eigsh()` with the Hamiltonian of an active site of a GPCR (G protein-coupled receptor) molecule. This demonstrates scaling behavior on a problem with a much larger computational subspace than the 22-qubit demo above.



Saving Hamiltonian Data

To run the multi-GPU solver, we first save the Hamiltonian data (pauli words, coefficients) and the sampled basis states so they can be loaded by each MPI rank.

```
[8]: # # uncomment this to save the data

# cumulative_subspace_results = cumulative_results[max_k-1]
# basis_states = get_basis_states_as_array(cumulative_subspace_results, num_spins)

# with open('skqd_data.pkl', 'wb') as f:
#     pickle.dump({
#         'basis_states': basis_states,
#         'pauli_words': pauli_words,
#         'hamiltonian_coefficients': hamiltonian_coefficients_numpy,
#     }, f)
```

Running the Distributed Solver

Save the code below as `distributed_postprocessing.py` and launch it with MPI. For example, on a system with 8 GPUs:

```
mpirun -np 8 --bind-to none --allow-run-as-root python distributed_postprocessing.py
```

```
[9]: """
Multi-GPU SKQD Eigenvalue Solver

Usage:
    Save the code below in a distributed_postprocessing.py file
    Execute with the command below, for example on a system with 8 GPUs:
    mpirun -np 8 --bind-to none --allow-run-as-root python distributed_postprocessing.py
    """

import numpy as np
import cupy as cp
from skqd_src.pre_and_postprocessing import *
from mpi4py import MPI
import pickle
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

n_devices = cp.cuda.runtime.getDeviceCount()
my_gpu = rank % n_devices
cp.cuda.Device(my_gpu).use()

tile_size = 32

#uncomment this to load the data
# with open('skqd_data.pkl', 'rb') as f:
#     data = pickle.load(f)
#     basis_states = data['basis_states']
#     pauli_words = data['pauli_words']
#     hamiltonian_coefficients_numpy = data['hamiltonian_coefficients']

subspace_dimension = basis_states.shape[0]

ham_data = prepare_hamiltonian_data(basis_states, pauli_words, hamiltonian_coefficients_numpy)
cp.cuda.Device(my_gpu).synchronize()

nccl_comm_obj = None
if size > 1:
    from cupy.cuda import nccl
    if rank == 0:
        nccl_id = nccl.get_unique_id()
    else:
        nccl_id = None
    nccl_id = comm.bcast(nccl_id, root=0)
    nccl_comm_obj = nccl.NcclCommunicator(size, nccl_id, rank)

np.random.seed(42)
v0_np = np.random.random((subspace_dimension,)).astype(np.float64)
v0_np = v0_np / np.linalg.norm(v0_np)
v0 = cp.asarray(v0_np, dtype=cp.complex128)

cp.get_default_memory_pool().free_all_blocks()

ncv = min(20, subspace_dimension - 1)

eigenvalues, = distributed_eigsh(
    ham_data, rank, size, nccl_comm_obj,
    k=1, which='SA', ncv=ncv, v0_full=v0,
```

```
tile_size=tile_size, return_eigenvectors=False)

eigenvalue = eigenvalues[0]
cp.cuda.Device(my_gpu).synchronize()
eigenvalue_real = float(cp.asnumpy(eigenvalue).real)

MPI.Finalize()
```

Summary

In this notebook we demonstrated the **Sample-Based Krylov Quantum Diagonalization (SKQD)** algorithm using CUDA-Q:

1. **Quantum phase:** Generated a sequence of Krylov states $U^k|\psi_0\rangle$ via Trotterized time evolution on a 22-qubit Heisenberg model, and sampled each state in the computational basis.
2. **Classical post-processing:** Projected the Hamiltonian onto the computational subspace defined by the measurement outcomes, and extracted the ground state energy via eigenvalue solvers.
3. **Two solver approaches:** Compared an explicit GPU-accelerated CSR sparse matrix solver with a matrix-free Lanczos iteration (`distributed_eigsh`), showing both converge to the same result.
4. **GPU acceleration:** Demonstrated significant speedup of the classical post-processing when using GPU-vectorized operations compared to CPU.
5. **Multi-GPU scaling:** Showed how the matrix-free Lanczos solver scales across multiple GPUs using NCCL for distributed computation.